Functional & Performance Testing Template

Model Performance Test

Date	06 November 2026
Team ID	Individual (Abdul Khadarvali)
Project Name	Employee Raise Issue – Record Producer & Service Portal Integration Guide (Modeled on ServiceNow Capstone Reference Repository)
Maximum Marks	4 Marks

Test Scenarios & Results

Test Case ID	Scenario (What to test)	Test Steps (How to test)	Expected Result	Actual Result	Pass/Fail
FT-01	**Scoped Application & Custom Table Schema Verification**	1. Navigate to **All menus → App Engine → ServiceNow Studio** and verify the **Employee Center** scoped application. 2. Open **System Definition → Tables**, locate **Employee raises issue**, verify **Auto number** is enabled in Controls, and open **Form Design** to inspect all 8 fields.	The custom table **Employee raises issue** is active inside the **Employee Center** scope with all 8 fields configured: *Number*, *Requester*, *Category*, *Subcategory*, *Short description*, *State*, *Priority*, and *Assignment Group*.	All 8 custom fields, reference lookups (*sys_user*, *sys_user_group*), choice lists, and auto-numbering controls rendered and saved as configured.	**Pass**
FT-02	**Category & Dependent Subcategory Choice Filtering**	1. Open a new record on the **Employee raises issue** form or **Raise Employee Issue** Record Producer. 2. Select *Network*, *Hardware*, *Software*, and *Access* sequentially in the **Category** field. 3. Inspect the filtered options displayed in **Subcategory** for each selection.	Selecting *Network* displays **VPN**; selecting *Hardware* displays **Laptop**; selecting *Software* displays **Server**; selecting *Access* displays **Forgot password** via Dictionary Dependency (*Dependent = Category*).	The **Subcategory** dropdown dynamically filtered its choices to match the exact *Dependent value* configured for each selected **Category**.	**Pass**
FT-03	**Mandatory Short Description UI Policy Validation**	1. Open the **Employee raises issue** form. 2. Set **State** to *New* and leave the **Short description** field empty. 3. Attempt to submit or save the record, then enter a valid summary and resubmit.	UI Policy (*When the state changes to new then short description is mandatory*) marks **Short description** with a mandatory asterisk and blocks form submission when empty, allowing save once populated.	Form submission was blocked with a mandatory field alert when blank and succeeded immediately once **Short description** was filled in.	**Pass**

Test Case ID	Scenario (What to test)	Test Steps (How to test)	Expected Result	Actual Result	Pass/Fail
FT-04	**Read-Only Number Field UI Policy Enforcement**	1. Open any new or existing record on the **Employee raises issue** form. 2. Attempt to click into or manually edit the auto-generated value inside the **Number** field.	UI Policy (*Number field will read only*) enforces *Read only = true* on the **Number** field across all states so users cannot alter system-generated ticket numbers.	The **Number** field remained locked and non-editable across both new and existing records, preserving auto-numbering integrity.	**Pass**
FT-05	**Service Portal (employee_request) & 6*6 Grid Widget Rendering**	1. Navigate to **Service Portal → Portals** and launch the **Requesting portal** at URL suffix /employee_request. 2. Inspect the **Employee Center Portal Page** (*employee_center_portal_page*) layout in the browser.	The portal loads with the *Employee Center Coral Theme* and displays the **6*6** two-column layout: **Commercial widget** (gradient service overview card) on the left and **Link redirect** card on the right.	Both custom widgets rendered side by side in the **6*6** container with accurate tri-color CSS linear-gradient styling and responsive alignment.	**Pass**
FT-06	**Record Producer Redirect & End-to-End Ticket Creation**	1. On /employee_request, click **Raise Ticket →** inside the **Link redirect** widget. 2. Fill in the 5 variables on **Raise Employee Issue**: Order 100 (*What is the Requester name*), Order 200 (*Which Category*), Order 300 (*What is Subcategory*), Order 400 (*Which State is this issue*), and Order 500 (*Short description*). 3. Click **Submit** and check the **Employee raises issue** table.	Clicking the card triggers *c.openRecordProducer()* via *$location.url()* to open item *sc_cat_item* (*ae2cd61183433210de6fc590ceaad3a4*) and inserts a new mapped record into the **Employee raises issue** table.	The widget redirected to the **Raise Employee Issue** catalog item and created a complete ticket record with all 5 variable values mapped to their target table columns.	**Pass**
PT-01	**Service Portal & Custom Widget Load Response Time Test**	1. Open browser developer tools (Network & Performance tabs). 2. Load /employee_request across multiple fresh sessions and measure page and widget render times.	The **Requesting portal** homepage, *Employee Center Coral Theme* assets, **Commercial widget**, and **Link redirect** widget render within **2 seconds**.	Portal page and both **6*6** custom widgets rendered in an average of **1.1–1.5 seconds** with zero layout shift or script errors.	**Pass**

Test Case ID	Scenario (What to test)	Test Steps (How to test)	Expected Result	Actual Result	Pass/Fail
PT-02	**Record Producer Redirect & Ticket Submission Speed**	1. Click **Raise Ticket →** on the portal to trigger the AngularJS *$location.url()* redirect. 2. Populate all 5 Record Producer variables (Orders 100–500) and click **Submit**, measuring round-trip transaction time.	Client-side navigation to *sc_cat_item* and server-side record insertion into **Employee raises issue** complete within **3 seconds**.	Client redirect completed in under **0.6 seconds** and Record Producer submission inserted the target table record within **1.4–1.8 seconds**.	**Pass**
PT-03	**Repeated Consecutive Ticket Submissions Load Test**	1. Submit 10 consecutive employee issue requests across all 4 categories (*Network*, *Hardware*, *Software*, *Access*) via the **Raise Employee Issue** Record Producer. 2. Inspect the **Employee raises issue** table list view.	The system processes all consecutive submissions without lag, generating unique sequential auto-numbered ticket IDs and preserving accurate field mappings.	All 10 consecutive submissions succeeded without latency spikes, duplicate numbers, or dropped field mappings in the **Employee raises issue** table.	**Pass**

Phase-Wise Verification & Quality Gate Summary

Implementation Phase	Target Artifact / Component	Verification Criteria & Test Coverage	Quality Status
Phases 1 & 2: Data Model	**Employee Center** Scope & **Employee raises issue** Table	Verified scoped isolation in ServiceNow Studio, *Auto number* controls, and all 8 custom columns via Form Design (**FT-01**).	**Verified (100%)**
Phase 3: Form Governance	UI Policies & Subcategory Dictionary Dependency	Verified mandatory *Short description* when *State is New*, read-only *Number* field lock, and 4 category-to-subcategory mappings (**FT-02, FT-03, FT-04**).	**Verified (100%)**
Phase 4: Service Catalog	**Raise Employee Issue** Record Producer	Verified catalog item under *Service Catalog → Service* and 1:1 field mapping across Orders 100, 200, 300, 400, and 500 (**FT-06**).	**Verified (100%)**
Phases 5–7: Portal & Load	**Requesting portal** (/employee_request) & Widgets	Verified *6*6* grid layout, *Commercial widget* CSS gradient, *Link redirect* AngularJS controller redirect, and sub-2s response time (**FT-05, PT-01–PT-03**).	**Verified (100%)**